LLM-Driven Evolutionary Search: Evolving packing programs

Can we improve the programs that find good circle packings?

ENGR 689 · Chapter 10 · Texas A&M University

Circle packing: the problem

What are we trying to arrange?

Place **26 or 32 circles** inside a square with side length 1. Choose each circle's **center** (its location) and **radius** (half its width). The radii may differ.

Every circle must stay inside the square. Circles may touch each other and the boundary, but they must **not overlap**.

A valid packing satisfies these rules for every circle. The number of circles stays fixed within each problem.

Circle packing: the goal

Which valid packing is better?

Maximize the **sum of the radii**: $S = r_1 + \cdots + r_N$, where N is the number of circles. Compare packings with the same N.

Four-circle illustration, with both arrangements valid:

Arrangement	Radius of each circle	Sum of radii
A	0.15	0.60
B	0.25	**1.00**

B has the larger score. Circle positions limit how large the radii can be. We need a strategy to search for better packings.

Our strategy: evolve packing programs

How can we improve the method that finds the packings?

Run several Python programs to produce packings. Check their circles and record their scores. Keep useful programs as **parents**.

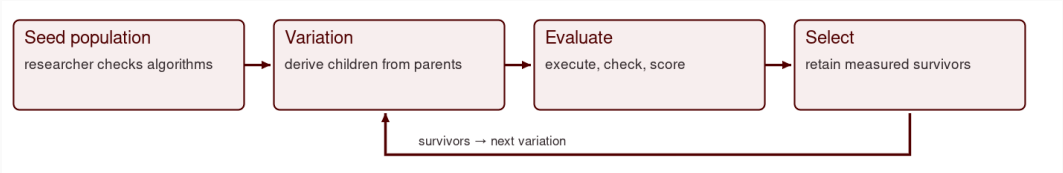


Figure 1: Initialize once; evaluate, select and mutate repeatedly.

Luna writes changes to the parent programs. We test each new program before deciding whether to keep it.

Five choices define the search

What must we define before running this search?

$(\mathcal{A}, f, \mu, \sigma, P_0)$: allowed programs, measured performance (**fitness**), how programs change (**mutation**), which programs we keep (**selection**), and the starting group (**initial population**).

$$P_{t+1} = \sigma(P_t \cup \{\mu(a) : a \in \sigma(P_t)\}) .$$

Choose parents $\rightarrow$ mutate $\rightarrow$ pool parents and children $\rightarrow$ select survivors.

New programs compete with the parents. A worse new program does not have to replace a useful parent.

Classical variation

How can we vary a program without understanding why it works?

Point mutation: change a move scale, for example $0.04 \to 0.02$. **Subtree replacement:** replace an acceptance expression. **Crossover:** inherit code from two parents.

These edits create hypotheses. The evaluator determines whether each resulting program still runs and improves its measured fitness.

Classical variation changes structure; selection supplies direction. Small source edits can have large behavioral effects.

A language model writes the mutation

What can a mutator change when it reads the parent code and its results?

1. Retain the parent's center search and radius allocator.
2. Propose a different move scale, center subset, or acceptance schedule.

3. Luna returns edited **Python source** and a hypothesis.
4. An isolated interpreter executes it; an independent checker evaluates the returned circles.

Luna implements the proposed code change; Python executes it. A reasoned edit becomes evidence only after evaluation.

Context joins the two course spines

Which evidence should the next mutation see?

Parent source **and scores**; successful and failed attempts; task constraints and useful domain knowledge; population similarity and remaining budget.

Evaluation becomes fitness. Context engineering drives mutation. The next request carries the previous round's actual results.

External memory makes the next proposal informed. A plausible explanation does not replace a measured trial.

Elitism protects measured progress

If a mutation is worse, must the current champion disappear?

Pool parents with the new child. Retain the best complete programs under the declared fitness rule.

For a maximized quality, elitism makes the observed best stay level or improve **under comparable evaluation conditions**.

A failed mutation need not erase a useful parent. Preserve the best valid packing separately from program fitness.

Selection pressure and population collapse

Why can keeping only the highest scores make search stagnate?

Top-k: rank everyone and retain the best k. **Tournament:** sample a group and retain its best; smaller tournaments can preserve more alternatives.

Collapse: several survivors become small rewrites of the same search. Current fitness does not reveal every parent's future descendants.

Strong selection exploits evidence but can discard useful routes. Our top-three rule does not guarantee diversity.

Feasibility comes before fitness

Could a flawed scorer reward an invalid packing?

Flawed scorer: sum radii without checking overlap. A valid pair scores **0.50**; an overlapping pair scores **0.60** and incorrectly wins.

Corrected evaluator: check every wall and pair, then score. Reject invalid or incomplete trials.

$$f(a \mid B_t) = -\operatorname{median}_{j=1,2,3} S(a; z_{t,j}).$$

Compare programs within one frozen packing bank B_t.

Close the scoring loophole; then test frozen programs on unseen starts.
Feasibility and transfer are separate checks.

Preserve different search behaviors

How can useful alternatives survive beside the current champion?

Islands: evolve separate populations, with occasional migration. **MAP-Elites:** retain the best program in each behavior bin, such as one-center versus coordinated moves.

Novelty search: favor behaviors unlike those already seen. Different source text alone does not establish different search behavior.

Preserve several ways to search. These are possible extensions; our top-three controller does not implement a diversity archive.

Exploration, exploitation and cost

When should we refine, explore—or stop spending?

Exploit: refine. **Explore:** change methods. **Stop:** respect the cap.

Hypothetical policies, same test conditions:

Policy	Median sum	Numerical seconds
A	2.40	1
B	2.41	5

Illustrative utility $S - 0.01t$ **favors A:** $2.39 > 2.36$**.** Our demo reports costs separately.

Demo 1: follow three mutations

Which part of the parent survives, and does the changed program earn its place?

Evaluate the initial population → inspect parent source and context → send Luna round 1 → read the actual code diff and verified packing.

Repeat for rounds 2 and 3. Inspect inherited packing-bank IDs, complete trial scores, failures and survivor decisions.

Follow the real chain: source → execution → checked geometry → fitness → selection → next context.

Read the first run's evidence

What improved: one packing, the program's measured fitness, or both?

A witness proves one feasible construction. Three complete trials rank a program on one declared bank. A source diff explains what changed; it does not prove an improvement.

Keep construction progress and program comparison separate. Report the actual outcome, calls and numerical effort.

Demo 2: the 32-circle problem

Does the same program-evolution mechanism still work when the number of circles changes?

Use $N = 32$, the second unit-square case in AlphaEvolve Appendix B.12. Keep the objective, checker, trial seeds, source interface and numerical limits declared.

Change the problem instance, then inspect adaptation. A higher raw sum for 32 circles is not evidence of a better algorithm than the 26-circle run.

Demo 2: inspect adaptation and limits

Does Luna change its search after seeing the 32-circle trial evidence?

Evaluate 32-circle parents → send three gated requests → inspect actual source edits, verified packings and the feedback entering each next request.

Keep invalid proposals, incomplete executions and repeated ideas visible. Stop cancels owned work; replay remains explicitly recorded.

The mechanism is testable even when improvement is absent. Evidence decides which proposed programs survive.

Name the framework we observed

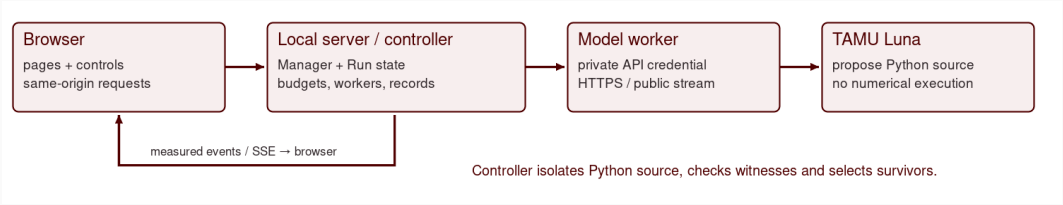
Can you identify every Chapter 10 component in the live record?

$\mathcal{A}$: Python packing programs. f: verified current-bank trial fitness. μ: Luna's code mutation. σ: complete-program selection. P_0: checked initial programs.

Context proposes; external evaluation measures; selection preserves. For your research problem, define these five choices before spending search budget.

Appendix A1: One server coordinates the experiment

Who owns the population, execution and measurements?



The controller owns the loop. Only the model worker contacts TAMU; a separate isolated worker executes generated Python.

Appendix A2: Request, context and source response

What crosses the model API boundary?

Send the task, parent source, current-bank trial results, history and limits. Authorization is a separate HTTPS header.

Return JSON: `round_id`, `parent_ids`, `hypothesis`, `source`. The source defines `solve(initial, seed, steps)`.

A source string is executable code after admission. The hypothesis is an explanation, never the authoritative score.

Appendix A3: Execution, checking and feedback

When can a returned program become a survivor?

Validate source $\rightarrow$ isolated trials $\rightarrow$ verify every circle and pair $\rightarrow$ aggregate three complete scores $\rightarrow$ select within the current packing bank.

Keep exact request/response records, valid witnesses and failures. The next context carries the measured history.

Execution turns a proposal into evidence. Live LLM, numerical-only and recorded replay remain distinct throughout.